Parcial 2 - Sistemas Operativos: Concurrencia y Sincronización

Curso: Sistemas Operativos
Tema Central: Concurrencia, Hilos (Threads), Exclusión Mutua, Sincronización y Condiciones de Carrera.
Modalidad: Los estudiantes deben seleccionar *una (1)* de las dos alternativas propuestas a continuación para su desarrollo e implementación.

Instrucciones Generales

- El proyecto debe ser implementado en lenguaje C o C++ bajo entorno POSIX/Linux.
- Se debe hacer un uso explícito y correcto de primitivas de concurrencia y sincronización (hilos pthread, mutex, variables de condición y/o semáforos).
- No se permite el uso de librerías de alto nivel que resuelvan o abstraigan automáticamente la sincronización requerida.

Condiciones de Entrega y Sustentación

- **Modalidad de Trabajo:** Se permite trabajar en equipos de hasta cinco (5) personas.
- **Plataforma y Evidencias de Entrega:** La entrega se realiza a través de EAFIT Interactiva. Cada miembro del equipo debe publicar individualmente las evidencias del trabajo realizado por el equipo en la plataforma.
- **Fecha Límite de Entrega:** La fecha máxima de entrega es en la semana 12 de clase.
- **Consultas y Asesorías:**

    - Se dispondrá de tres (3) horas en clase dedicadas exclusivamente a consultas.
    - La asistencia presencial a estas sesiones de consultoría en clase es de carácter obligatorio antes de poder solicitar cualquier asesoría adicional por Microsoft Teams.
- **Sustentación:**
    - La sustentación podrá realizarse mediante video grabado o de manera presencial en la oficina 19-424 (previa solicitud y agendamiento de reunión).
    - En la sustentación, el equipo debe presentar y defender rigurosamente los argumentos desde la perspectiva de la ingeniería para la solución del problema seleccionado.

Alternativa 1: Compresor de Archivos Huffman Concurrente Integrado al Editor

1. Descripción General

Esta alternativa consiste en diseñar e implementar un módulo de compresión y descompresión basado en el algoritmo de **Codificación de Huffman Concurrente**, e integrarlo como una funcionalidad en segundo plano (asíncrona) dentro del editor de texto desarrollado en el Parcial 1.

2. Requerimientos Técnicos y de Concurrencia

- **División y Procesamiento Paralelo:** El archivo debe fragmentarse en bloques (chunks) que son procesados de forma concurrente por un pool de hilos trabajadores (*worker threads*).
    - *Fase de Conteo de Frecuencias:* Múltiples hilos leen bloques del archivo en

paralelo y acumulan las frecuencias de caracteres utilizando sincronización adecuada o reducción local.
    - *Fase de Compresión / Codificación:* Tras construir el árbol de Huffman global, la codificación de cada bloque se realiza en paralelo.
- **Sincronización y Ensamblado:** Un hilo coordinador o cola de salida sincronizada debe garantizar que los bloques comprimidos se escriban en el archivo de salida en el orden secuencial correcto, sin generar bloqueos activos ni inconsistencias.
- **Integración con el Editor (Parcial 1):**
    - La compresión y descompresión no deben bloquear la interfaz de usuario ni la interacción del editor. Se debe ejecutar como una tarea en segundo plano (*background worker*).
    - El editor debe mostrar el estado o progreso de la compresión/descompresión en tiempo real (por ejemplo, barra de progreso o porcentaje completado).
    - Se deben manejar condiciones de carrera si el usuario intenta editar o guardar el archivo mientras está siendo comprimido/descomprimido.

3. Rúbrica de Evaluación - Alternativa 1 (100 puntos)

Criterio	Descripción	Puntaje Máximo
Concurrencia en Huffman	Paralelización efectiva del conteo de frecuencias y compresión por bloques usando hilos. Manejo ordenado de la salida.	25 pts
Mecanismos de Sincronización	Uso riguroso de mutexes, semáforos y/o variables de condición. Ausencia total de condiciones de carrera, interbloqueos y espera activa (busy	25 pts

Criterio	Descripción	Puntaje Máximo
	waiting).	
Integración con el Editor	Ejecución en segundo plano sin congelar la UI/editor; sincronización de acceso al archivo/buffer ante modificaciones concurrentes.	20 pts
Corrección Algorítmica	Los archivos comprimidos se descomprimen de manera idéntica al original (integridad comprobada mediante checksum/md5/diff).	15 pts
Manejo de Errores y Recursos	Liberación adecuada de memoria dinámica, cierre de descriptores de archivo, join/detach limpio de hilos y control de excepciones/señales.	15 pts

Alternativa 2: Reproductor de Audio y Gestión Concurrente de Lista de Reproducción

1. Descripción General

Esta alternativa consiste en construir o extender un reproductor de audio en línea de comandos o interfaz básica CLI, donde el foco central es el diseño concurrente del motor de reproducción desacoplado de la gestión dinámica y concurrente de la lista de reproducción (*playlist*).

2. Requerimientos Técnicos y de Concurrencia

- **Arquitectura Productor-Consumidor / Desacoplamiento:**
    - *Hilo Lector/Decodificador (Productor):* Lee archivos del archivo de audio actual y los deposita en un búfer circular compartido.
    - *Hilo de Salida de Audio (Consumidor):* Extrae muestras del búfer circular a la frecuencia de muestreo requerida y las envía al el programa que reproduce el audio (ALSA, PulseAudio, libao o similar).
    - Sincronización estricta del búfer circular para prevenir subdesbordamiento (*buffer underrun*) o sobrellenado sin incurrir en espera activa.
- **Gestión Concurrente de la Lista de Reproducción (Playlist):**
    - La lista de reproducción es una estructura compartida sujeta al problema de **Lectores-Escritores** o colas concurrentes seguras.
    - Mientras el hilo de reproducción lee la pista actual y prepara la siguiente pista (lectura), el usuario puede concurrentemente: agregar canciones, eliminar canciones intermedias, reordenar la lista, pausar, saltar a la siguiente pista o limpiar la cola (escritura/modificación).
    - Se deben utilizar bloqueos de lectura/escritura (pthread_rwlock) o mutexes con variables de condición para evitar accesos inválidos a nodos o memoria liberada.
- **Control de Reproducción y Señalización:** Control asíncrono e inmediato de eventos como *Play*, *Pause*, *Stop*, *Next* y *Previous*, respondiendo sin latencia apreciable ni corromper el estado del decodificador ni de la lista.

3. Rúbrica de Evaluación - Alternativa 2 (100 puntos)

Criterio	Descripción	Puntaje Máximo
Gestión Concurrente de Playlist	Estructura compartida segura para lectura/escritura. Capacidad de modificar, reordenar y consultar la cola mientras se	25 pts

Criterio	Descripción	Puntaje Máximo
	reproduce audio sin fallos de segmentación ni estados corruptos.	
Búfer de Audio (Productor-Consumidor)	Diseño e implementación de un búfer circular sincronizado entre decodificación y salida de audio con semáforos/variables de condición; sin chasquidos por underrun evitable.	25 pts
Control Asíncrono de Eventos	Manejo no bloqueante de comandos de usuario (Play, Pause, Skip, Next/Prev) con señalización inmediata entre hilos.	20 pts
Robustez y Ausencia de Deadlocks	Código libre de condiciones de carrera, interbloqueos, memory leaks y manejo consistente al finalizar pistas o cancelar la lista.	15 pts
Diseño Modular y Buenas Prácticas	Separación clara de responsabilidades (UI/CLI, motor de audio, cola de reproducción), documentación del protocolo de concurrencia y limpieza de código.	15 pts